

اصل (OCP (Open/Closed Principle می‌گوید:

کلاس‌ها باید برای توسعه باز باشند (Open for extension) ولی بسته برای تغییر (Closed for modification)

یعنی شما باید بتوانید رفتار جدید اضافه کنید بدون اینکه کلاس اصلی را تغییر دهید.

✗ مثال نقض OCP در C#

```
public class TaxCalculator
{
    public double CalculateTax(string type, double amount)
    {
        if (type == "Food")
        {
            return amount * 0.05;
        }
        else if (type == "Electronics")
        {
            return amount * 0.15;
        }
        else
        {
            return 0;
        }
    }
}
```

● مشکل:

-اگر بخواهیم مالیات نوع جدید اضافه کنیم (مثلا Books) باید خود کلاس را تغییر دهیم.

-این یعنی نقض OCP، چون کلاس برای تغییر بسته نیست و برای اضافه کردن رفتار جدید مجبوریم آن را تغییر بدهیم.

✓ اصلاح مطابق OCP با استفاده از Interface و Polymorphism

```
// تعریف یک اینترفیس برای محاسبه مالیات
public interface ITax
{
    double Calculate(double amount);
}

// پیاده‌سازی برای غذا
public class FoodTax : ITax
{
    public double Calculate(double amount)
    {
        return amount * 0.05;
    }
}

// پیاده‌سازی برای لوازم الکترونیکی
public class ElectronicsTax : ITax
{
    public double Calculate(double amount)
    {
        return amount * 0.15;
    }
}

// سیستم محاسبه مالیات
public class TaxCalculator
{
    public double CalculateTax(ITax tax, double amount)
    {
        return tax.Calculate(amount);
    }
}
```

● حالا:

برای اضافه کردن نوع جدید مالیات، کافی است یک کلاس جدید که ITax را پیاده‌سازی کند بسازیم، بدون تغییر در TaxCalculator.

مثال اضافه کردن مالیات کتاب:

```
public class BookTax : ITax
{
    public double Calculate(double amount)
    {
        return amount * 0.08;
    }
}
```

نتیجه: 💡

-کلاس اصلی TaxCalculator تغییر نکرده و بسته است.
-می‌توانیم هر تعداد مالیات جدید اضافه کنیم و کلاس باز برای توسعه است.